Memory Architecture for AtmoClaw: State of the Art and Recommendations

Executive Summary

The AI agent memory landscape in early 2026 has converged on a clear architectural consensus: the field has moved past naive chat-history buffers and split into two distinct problem classes — **personalization memory** (remembering user preferences and session context) and **institutional memory** (accumulating domain knowledge that compounds over time). AtmoClaw needs the latter. The documents provided already identify the right instincts — Obsidian as a knowledge layer, STAC as a data catalog, skepticism of framework lock-in — but underestimate how fast the tooling has matured and where the real architectural leverage now sits.

This brief synthesizes current research, production experience reports, and framework comparisons to deliver opinionated recommendations for building AtmoClaw's memory system. The core argument: **your existing plan is 80% right, but the missing 20% — specifically, a temporal knowledge graph for agent-generated findings and sleep-time memory consolidation — is where the compounding value lives.**

Where the Field Has Converged

The Three-Memory Taxonomy Is Now Standard

The academic consensus, formalized in a comprehensive survey from December 2025, identifies three functional categories of agent memory that map directly to AtmoClaw's needs:

Memory Type	What It Stores	AtmoClaw Equivalent	Current Best Tool
Factual memory	Domain knowledge, entity relationships, structured facts	Simulation findings, physical constants, model configurations	Knowledge graph (Graphiti/Cognee) + Obsidian
Experiential memory	Past episodes, outcomes, what worked and what didn't	Prior analysis sessions, experiment results, verification outcomes	Structured logs + vector retrieval
Working memory	Active task context, current reasoning state	Current research session, in-progress analysis	LLM context window + CLAUDE.md

The key insight from the Vectorize framework comparison and production reports from 2026 is that **most teams over-invest in working memory (prompt engineering) and under-invest in factual and experiential memory.** AtmoClaw's documents already identify this — the "living knowledge base" vision is exactly right. The question is implementation.

Knowledge Graphs Are Winning Over Pure Vector Search

The field has clearly moved toward **hybrid architectures** that combine vector similarity search with knowledge graph traversal. Pure vector databases (Pinecone, Qdrant, ChromaDB) remain excellent for semantic search over unstructured text, but they fundamentally cannot perform multi-hop reasoning — connecting Entity A to Entity C when the only stored relationship is A→B and B→C.

For AtmoClaw, this matters critically. When an agent needs to answer "what do we know about the sensitivity of hurricane intensity to CCN concentration in the outer rainbands?", that answer lives across dozens of simulation runs, analysis notes, and experiment summaries connected by entity relationships. A vector similarity search might find relevant documents, but a knowledge graph can **traverse the actual chain of reasoning** across experiments.

The benchmark data supports this convergence. Graph-based approaches achieve 92% recall and 88% precision versus 85% recall and 75% precision for pure RAG, at the cost of higher implementation complexity and maintenance burden (3-4 person-months/year versus 0.5 for RAG, per the same analysis).

Sleep-Time Compute Is the Next Major Paradigm

The most important emerging pattern is sleep-time compute, pioneered by the Letta (formerly MemGPT) team and validated in a peer-reviewed paper. The idea: agents should process and consolidate memory **during idle periods**, not during active interaction.

Anthropic has already shipped this in production — Claude Code's Auto Dream feature runs background memory consolidation that converts relative dates to absolute timestamps, deletes contradicted facts, prunes stale entries, and merges duplicates. One observed cycle processed 913 sessions in about 8-9 minutes.

For AtmoClaw, this pattern maps directly: after every simulation run completes, a background agent should consolidate findings into the knowledge base — not during the scientist's interactive session, but asynchronously. This is the "post-simulation hook" concept from the Obsidian document, but with LLM-powered synthesis rather than simple metadata logging.

The Memory Framework Landscape: What Actually Matters

Framework Comparison for AtmoClaw's Use Case

Based on extensive research across the current ecosystem, here is how the major memory frameworks map to AtmoClaw's specific requirements:

Framework	Architecture	Institutional Knowledge	Temporal Awareness	Self-Hostable	AtmoClaw Fit
Graphiti (open-source engine behind Zep)	Temporal knowledge graph	Strong	Best in class	Yes	High — entity/relationship tracking with time validity
Cognee	KG + Vector, pipeline-based	Strong	Moderate	Yes (fully local)	High — 30+ data source connectors, multimodal
Letta	OS-inspired tiered memory	Strong (agent-managed)	Moderate	Yes	Medium — powerful but it's a full runtime, not a library
Mem0	Vector + Graph (graph requires Pro)	Moderate (Pro only)	Weak	Yes	Low — optimized for personalization, not research memory
Hindsight	Multi-strategy hybrid	Strong	Moderate	Yes	Medium — strong retrieval but newer project

Sources: Vectorize comparison, Letta developer forum, Zep paper

Why Mem0 Is Not the Right Primary Choice

Mem0 has the largest community (~48K GitHub stars) and is the easiest to set up, but it is fundamentally optimized for personalization memory — remembering that "User A prefers dark mode" or "this agent should respond formally." Its graph features, which are what AtmoClaw would actually need, are gated behind a $249/month Pro tier. Independent evaluation shows 49.0% on LongMemEval, significantly below other dedicated memory systems. The Obsidian-vs-Mem0 document already reached the right conclusion: Mem0 is a sidecar at best, not a primary knowledge layer.

Why Graphiti/Zep Deserves Serious Attention

Graphiti — the open-source temporal knowledge graph engine behind Zep — is the most architecturally aligned tool for AtmoClaw's needs. Its core innovation is that every fact carries validity windows: when it became true and when it was superseded. This matters for scientific research. If simulation run #47 showed CCN sensitivity at one threshold, but run #142 with a corrected parameterization showed a different threshold, the knowledge base needs to track that evolution, not just store two contradictory facts.

Graphiti achieves 94.8% on the Deep Memory Retrieval benchmark (versus 93.4% for MemGPT), with latency improvements of up to 90% compared to baseline implementations. It is open-source (MIT-like license), self-hostable, and available as a Python library that runs on top of Neo4j.

The critical distinction: Zep Cloud (the commercial product) has deprecated its Community Edition for self-hosting. But Graphiti (the underlying engine) remains fully open-source and can

be self-hosted. For AtmoClaw, use Graphiti directly — not Zep Cloud.

What I Think You Should Do

The Recommended Architecture

Based on the research, I'm recommending a **three-layer memory architecture** for AtmoClaw that builds on the existing documents but diverges in important ways:

```
┌──────────────────────────────────────────┐
│          Layer 3: Working Memory          │
│  CLAUDE.md / System Prompts / Session State │
│    (Active context for current task)       │
└──────────────────────────────────────────┘
                    ↕
┌──────────────────────────────────────────┐
│        Layer 2: Knowledge Graph           │
│     Graphiti (temporal KG) + Obsidian      │
│  (Accumulated findings, entity relations,   │
│   experiment lineage, temporal evolution)   │
└──────────────────────────────────────────┘
                    ↕
┌──────────────────────────────────────────┐
│         Layer 1: Data Catalog             │
│        STAC + Zarr/Icechunk               │
│   (Where simulation data lives, metadata)   │
└──────────────────────────────────────────┘
```

Layer 1: STAC + Zarr/Icechunk (Keep As-Is)

The documents are right — STAC is the foundation. No changes to recommend here. The OGC Community Standard recognition in October 2025 and the Zarr-in-STAC best practices from the November 2025 community sprint confirm this is the correct long-term bet. STAC answers "where is the data and what does it contain."

Build this first. Everything else depends on it.

Layer 2: Graphiti + Obsidian (Modified from Current Plan)

Here is where I diverge from the current documents. The Obsidian-only plan is good but incomplete. Obsidian is excellent for **human-readable, human-editable research notes** — the lab notebook. But it lacks the structured query capabilities that an autonomous agent needs for multi-hop reasoning over thousands of experiments.

Use both, for different purposes:

	Obsidian	Graphiti
Primary consumer	Humans (scientists browsing, editing, annotating)	Agents (querying, traversing, synthesizing)

	Obsidian	Graphiti
Data model	Markdown files with YAML frontmatter, wikilinks	Nodes (entities) and edges (relationships) with temporal validity
Query model	Full-text search, tag filtering, Dataview SQL-like queries	Cypher graph queries, vector similarity, temporal interval queries
Strengths	Human-readable, git-friendly, scientists can correct agent interpretations	Multi-hop reasoning, temporal evolution tracking, structured entity resolution
Weaknesses	Cannot do multi-hop graph traversal, limited structured query	Not human-browsable, requires Neo4j infrastructure

The bridge: Every Obsidian note should have a STAC item ID in its frontmatter (as the current documents propose). Additionally, key findings extracted from Obsidian notes should be ingested into Graphiti as episodes, where they are automatically decomposed into entities, relationships, and temporal attributes.

Practical implementation:

1. Keep Obsidian as the human-facing knowledge base — the lab notebook that scientists browse, edit, and annotate
2. Add Graphiti as the agent-facing knowledge graph — the structured, queryable representation that agents traverse
3. Sync from Obsidian → Graphiti: when a new analysis note is created or updated in Obsidian, a background process ingests it into Graphiti
4. Agents query Graphiti first for structured relationships, fall back to Obsidian full-text search for broader context

This dual-store architecture is more complex than Obsidian alone, but it solves the fundamental problem: Obsidian cannot answer "what experiments have we run that tested CCN thresholds between 100 and 500 cm^{-3} and showed a nonlinear response in hurricane intensity, ordered by the date we discovered the relationship?" Graphiti can.

Layer 3: Working Memory via Claude Code / Claude Agent SDK

The Claude Code harness document is on target. Use CLAUDE.md for persistent project instructions, let auto memory handle session-to-session learning, and use MCP servers to expose domain tools.

The specific addition I recommend: **implement sleep-time consolidation.** After every simulation run completes, trigger a background agent (via the Claude Agent SDK's programmatic interface) that:

1. Reads the new simulation output metadata from STAC
2. Searches Obsidian and Graphiti for related prior work
3. Runs verification checks
4. Creates an Obsidian analysis note with findings

5. Ingests the findings into Graphiti as new episodes
6. Updates any contradicted or superseded facts in Graphiti with temporal validity markers

This is the "post-simulation hook" from the Obsidian document, upgraded with Letta-style sleep-time compute for proper memory consolidation.

What to Skip

Skip Letta as a runtime. Letta's OS-inspired memory architecture is intellectually elegant but it's a full agent platform, not a library. Adopting it means adopting their runtime, their agent loop, their deployment model. The Claude Agent SDK already provides the agent loop; you don't need two. The self-editing memory concept from MemGPT is valuable — implement it as a pattern (agents that read/write their own memory blocks) without adopting the Letta platform.

Skip Cognee for now. Cognee is strong for multimodal data ingestion (documents, images, audio), but AtmoClaw's primary data is structured simulation output and agent-generated analysis notes, not unstructured document corpora. Cognee's sweet spot is when you have 30+ heterogeneous data sources to ingest. AtmoClaw has a well-defined data pipeline (Zarr → STAC → analysis). Cognee adds complexity without proportional value here.

Skip LangChain memory modules. The agent-stack-choice document already makes this case well. LangChain/LangGraph memory (MemorySaver, SqliteSaver, PostgresSaver) is basic checkpointing — it stores conversation state, not structured knowledge. It's the equivalent of saving your chat history to a database. That's not what AtmoClaw needs.

The Build Sequence

This is the order I'd build in, accounting for both value delivery and dependency chains:

Phase 1: Foundation (Months 1-2)

1. **STAC catalog** — populated and maintained for all simulation runs
2. **Zarr/Icechunk tools** — MCP server wrapping data access functions
3. CLAUDE.md + **skills** — domain knowledge, verification procedures, common workflows
4. **Claude Code CLI** — scientists can ask questions interactively (this works immediately once 1-3 exist)

This delivers the "data analyst" from the data-analyst-vs-research-agent document. No memory system needed yet — the scientist is the memory.

Phase 2: Knowledge Accumulation (Months 3-4)

5. **Obsidian vault** — templates for run notes, analysis notes, experiment notes
6. **Post-simulation hooks** — auto-create Obsidian run notes with STAC metadata after every job
7. **Structured interaction logging** — append-only log of every agent interaction (question, data accessed, result)

8. **Claude Agent SDK** for programmatic / scheduled analysis

This is the lightweight knowledge layer. Not autonomous yet, but building the corpus that will become valuable.

Phase 3: Structured Knowledge Graph (Months 5-6)

9. **Graphiti deployment** — Neo4j + Graphiti, self-hosted
10. **Obsidian → Graphiti sync pipeline** — ingest analysis notes as episodes
11. **Agent retrieval from Graphiti** — MCP server exposing graph queries to Claude
12. **Sleep-time consolidation agent** — background memory processing after simulation runs

This is where the knowledge base starts compounding. The agent can now traverse the graph of prior experiments to inform new analysis.

Phase 4: Autonomous Research Capabilities (Months 7+)

13. **Cross-run learning** — agents query Graphiti to find related experiments before designing new ones
14. **Anomaly detection** — agents monitor incoming results against historical patterns in the knowledge graph
15. **Hypothesis generation** — agents propose follow-up experiments based on graph-traversal of accumulated findings

This is the Phase 3 vision from the PRD — autonomous research. Don't rush here. The knowledge graph needs critical mass before autonomous research becomes valuable.

The Vector Database Question

For the vector similarity search component (which Graphiti uses internally and which you'll want for semantic search over Obsidian notes), use **Qdrant**. The reasoning:

- Rust-based architecture delivering sub-millisecond queries with the lowest p99 latency in independent benchmarks
- Open-source with flexible deployment (bare-metal, Kubernetes, or managed cloud)
- Advanced metadata filtering with hybrid search (BM25 + vectors)
- SOC-2 compliance, RBAC, production-ready
- Avoids the cloud-only lock-in of Pinecone and the scaling limitations of ChromaDB

For a 16-person team running thousands (eventually tens of thousands) of simulation analyses, Qdrant's self-hosted option gives you full control over your data and avoids per-query cloud pricing.

Where I Disagree with the Current Documents

On "Defer the Knowledge Base"

The data-analyst-vs-research-agent document argues to wait for observable signals (scientists asking questions about forgotten runs, agents re-running analyses) before building the knowledge layer. I disagree. **The structured interaction log should start from day one.** Not a full knowledge graph — just an append-only record of every agent interaction. This is cheap to implement and impossible to backfill. When you eventually need the knowledge base, this log is the raw material. The document itself acknowledges this tension: "bolting on a knowledge layer later is harder than designing it in from the start."

Start the Obsidian vault and structured logging in Phase 1, not Phase 2. The marginal cost is low, and the optionality is high.

On Obsidian as the Sole Knowledge Layer

Obsidian is excellent for what it does — human-readable, human-editable, git-friendly markdown notes with bidirectional links. But it is not a database. Dataview gives you SQL-like queries over frontmatter properties, but it cannot do graph traversal, temporal queries, or vector similarity search. For an autonomous agent that needs to reason over thousands of experiments, Obsidian alone will hit a ceiling.

The Obsidian CLI also requires the Obsidian app to be running, which is a constraint for headless CI/server environments. The documents acknowledge this ("fall back to direct file I/O on the markdown files") but understate the friction. For agent-automated workflows, direct file I/O on markdown is fine for writing, but inadequate for the kind of structured queries the research agent will need.

On the Claude Agent SDK as the Only Orchestration

The Claude Code document makes a compelling case for the Claude Agent SDK as a thin orchestration layer. This is correct for Phase 1-2. But for Phase 3+ (autonomous research with background consolidation, parallel experiment management, cross-run learning), you will need durable execution guarantees that the Claude Agent SDK does not provide. The agent-stack-choice document's recommendation to evaluate PydanticAI + Temporal is prescient. PydanticAI now natively supports durable execution via Temporal, DBOS, and Prefect — this is architecturally closer to "reliable workflow system with agent steps" than to "agent framework with some workflow features."

For long-running autonomous research workflows (submit batch of simulations → wait hours → analyze results → update knowledge base → propose follow-ups), you need something that survives process restarts and infrastructure failures. The Claude Agent SDK alone does not provide this.

Summary of Concrete Tool Recommendations

Component	Recommended Tool	Rationale
Data catalog	STAC + Icechunk	Already planned, industry standard, OGC-recognized
Data access	Zarr via Icechunk	Already planned, non-negotiable for your data volumes
Human-facing knowledge base	Obsidian vault	Already planned — human-readable, git-friendly, CLI-accessible
Agent-facing knowledge graph	Graphiti (self-hosted) + Neo4j	Temporal entity tracking, multi-hop traversal, open-source
Vector database	Qdrant (self-hosted)	Best latency, open-source, hybrid search, production-ready
Working memory	CLAUDE.md + auto memory	Native to Claude Code, zero additional infrastructure
Agent loop	Claude Agent SDK	Already planned — thin, replaceable, MCP-native
Durable execution (Phase 3+)	PydanticAI + Temporal	For long-running autonomous workflows that need crash recovery
Interactive agent	Claude Code CLI	Already planned — scientists talk to it in terminal
Background consolidation	Sleep-time pattern via Claude Agent SDK	Async memory processing after simulation runs

The total additional infrastructure beyond what's already planned: Neo4j (for Graphiti), Qdrant (for vector search), and eventually Temporal (for durable execution). Everything else is software libraries, not services.

Final Take

The documents show clear, well-reasoned thinking about what AtmoClaw needs. The instincts are right: framework-agnostic tool layer first, explicit state machines over multi-agent theatrics, Obsidian over Mem0, STAC as the data foundation, Claude Agent SDK as a thin harness.

The gap is in the **structured knowledge layer** — the piece that turns thousands of individual analyses into a queryable, traversable graph of accumulated scientific understanding. Obsidian is half the answer (the human-readable half). Graphiti is the other half (the machine-queryable half). Together, they create the knowledge base that compounds.

The second gap is **sleep-time consolidation** — the background processing that converts raw agent interactions into structured knowledge without slowing down the scientist's interactive session. This is where the field is converging, and AtmoClaw is perfectly positioned to implement it: every simulation run is a natural trigger for background knowledge processing.

Build the data analyst first. Build the knowledge base from day one (even if lightweight). Add the knowledge graph when volume demands it. Add autonomous research when the

knowledge graph has critical mass. At every step, the tool layer is the durable asset — everything else is replaceable.